Project 2 — Cloud Dashboard Development

CPSY 300 • Phase 2: Deploy to Azure Cloud & Build Web Dashboard

Team: **Yuandong Yang**, **Ethan Bayarsaikhan**, **Justin Norman-Rance**

Reviewed: **2026-10-20** • Repository: github.com/YUANDONG-YANG/cloud-computing

100
marks: 6 categories (Deployment 20, Frontend 20, Viz 20, Integration 20, Cloud 10, Docs 10)

7,806
recipes across 5 diet types from All_Diets.csv

4
visualizations: grouped bar, doughnut, heatmap, scatter

Deployed
Live on Azure (canadacentral) on 2026-10-20; dashboard shows "API Connected"

> **Status of this report**
>
> The system was deployed to Azure on **2026-10-20** by running `Project2/deploy.sh` (region canadacentral). Function App: https://cpsy300-nutrition-api.azurewebsites.net. Dashboard: https://salmon-ocean-0ce94150f.4.azurestaticapps.net. Evidence is in section 7 (Deployment Evidence); every screenshot there was captured from the live resources and carries a date and time.

Table of Contents

1. Phase 1 to Phase 2 Transition
2. Cloud Architecture
3. Cloud Setup & Configuration
4. Dashboard Visualizations
5. Key Code Sections
6. Frontend-Backend Integration
7. Rubric & Status

8. Challenges & Solutions
9. Team Contributions

1. Phase 1 → Phase 2 Transition

Phase 2 replaces each local emulation from Phase 1 with the Azure service it stands in for. The Phase 2 column below describes what the committed code targets; the right-hand column states whether that target has actually been created in Azure yet.

Component	Phase 1 (Local)	Phase 2 (Cloud)	Live in Azure?
Function execution	Localhost (`func start`, Azure Functions Core Tools)	Azure Functions, Python 3.11, v2 programming model (`backend/function_app.py`)	Deployed 2026-10-20
Blob storage	Azurite emulator (`UseDevelopmentStorage=true`)	Azure Blob Storage, private container `datasets`	Deployed 2026-10-20
Dataset access	Local Azurite container	Private blob read with a connection string from app settings; no public blob access	Deployed 2026-10-20
Output	Local console / CSV files in `Project1/docs/results/`	JSON from `/api/insights` and `/api/recipes`, rendered as charts in the browser	Code complete
Frontend	N/A	Azure Static Web App serving `frontend/index.html`, `app.js`, `styles.css`	Deployed 2026-10-20
CI/CD	GitHub Actions: build, pytest, flake8, Docker + Azurite simulation	GitHub Actions runs lint and unit tests only (`project2-tests` job); Azure deployment is performed manually with `deploy.sh`	CI only
Goal	Build and test locally	Deploy, visualize and interact in the cloud	Deployed 2026-10-20

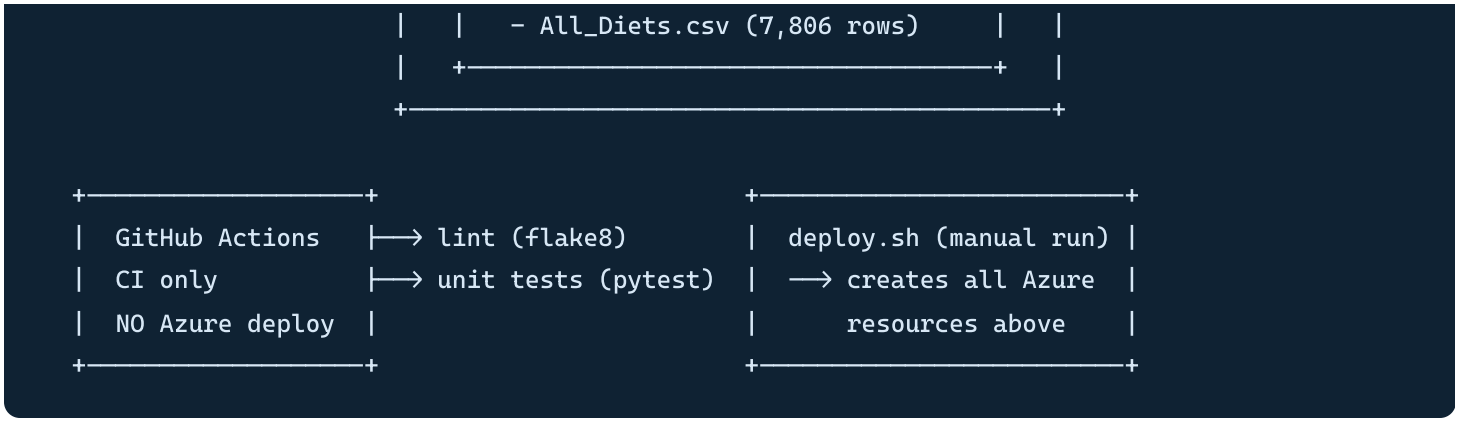
What the CI pipeline does and does not do

CI is split into one workflow per phase. `.github/workflows/deploy.yml` runs the Phase 1 `local-simulation` job: it builds the Docker image, runs `pytest` and `flake8`, and exercises the Azurite-based local simulation. `.github/workflows/project2-tests.yml` runs the Phase 2 `project2-tests` job: on Python 3.11 it installs `Project2/requirements-dev.txt`, runs `python -m pytest -q` against `Project2/tests/` (`test_function_app.py`, `test_nutrition.py`, `test_frontend_demo_data.py`), runs `flake8 .`, and syntax-checks the deployment script with `bash -n deploy.sh`. **Neither job deploys anything to Azure.** There is no `azure/functions-action` step, no `Azure/static-web-apps-deploy` step and no Azure credentials or publish profile in the workflow. Deployment is a deliberate manual step so that no subscription secrets are needed in CI.

2. Cloud Architecture

The application is a three-service serverless design. The browser loads the dashboard from the Static Web App, the dashboard calls the Function App over HTTPS, and the Function App reads the CSV from Blob Storage. Names, regions and settings below are taken from `Project2/deploy.sh`.





Architecture Components

Static Web App

Serves the dashboard as plain static files. Charts are drawn with Chart.js 4.4.1 plus a CSS-grid heatmap built in `app.js`. `staticwebapp.config.json` adds a navigation fallback and the `X-Content-Type-Options`, `X-Frame-Options` and `Referrer-Policy` headers.

Function App

Three anonymous HTTP GET routes registered with the Azure Functions v2 decorator model. Reads the CSV from Blob Storage, cleans it with the shared `nutrition.py` module, and returns JSON. The cleaned DataFrame is cached by blob ETag.

Blob Storage

A private `datasets` container holding `All_Diets.csv` (7,806 recipes). Public blob access is disabled and the minimum TLS version is 1.2. The connection string lives only in Function App settings.

GitHub Actions

Continuous integration only: `pytest`, `flake8` and a `bash -n` syntax check of `deploy.sh`. No Azure deployment job and no subscription secrets. Azure resources are created by running `deploy.sh` manually.

3. Cloud Setup & Configuration

Every command in this section is the one `deploy.sh` actually runs. The script is idempotent (`set -euo pipefail`, re-running updates existing resources) and runs in seven numbered steps. It has **executed against the Azure for Students subscription on 2026-10-20**.

3.1 Resource Group — step 1 of 7

> **Resource Group Configuration**
>
> All resources live in one resource group, so the whole project can be tracked and deleted as a unit.

```
RESOURCE_GROUP="cpsy300-nutritional-insights-rg"
LOCATION="canadacentral"
# Static Web Apps is only available in a handful of regions; canadacentral is not one.
SWA_LOCATION="eastus2"

az group create \
    --name "$RESOURCE_GROUP" \
    --location "$LOCATION" \
    --output none
```

3.2 Storage Account and dataset upload — step 2 of 7

Azure Blob Storage Setup

Storage account names are globally unique, so the script reuses an existing `cpsy300nutrition*` account in the resource group when re-run and otherwise generates `cpsy300nutrition$(openssl rand -hex 3)`. The container is **private**: public blob access is disabled at the account level and TLS 1.2 is the floor.

```
STORAGE_ACCOUNT="cpsy300nutrition$(openssl rand -hex 3)"   # e.g. cpsy300nutrition4f9a21
BLOB_CONTAINER="datasets"

az storage account create \
    --name "$STORAGE_ACCOUNT" \
    --resource-group "$RESOURCE_GROUP" \
    --location "$LOCATION" \
    --sku Standard_LRS \
    --kind StorageV2 \
    --min-tls-version TLS1_2 \
    --allow-blob-public-access false \
    --output none

CONN_STRING=$(az storage account show-connection-string \
    --name "$STORAGE_ACCOUNT" --resource-group "$RESOURCE_GROUP" \
    --query connectionString --output tsv)

az storage container create \
    --name "$BLOB_CONTAINER" \
    --connection-string "$CONN_STRING" \
    --output none

az storage blob upload \
    --container-name "$BLOB_CONTAINER" \
    --file "../Project1/data/All_Diets.csv" \
    --name "All_Diets.csv" \
    --connection-string "$CONN_STRING" \
    --overwrite \
    --output none
```

Secret handling: `$CONN_STRING` is never echoed. It is passed straight into the Function App settings, and the summary at the end of the script prints only the `az functionapp config appsettings list` command needed to retrieve it later.

3.3 Function App — steps 3 and 4 of 7

Azure Function App Deployment

A Linux Consumption-plan Function App on Python 3.11 and Functions runtime v4. The code is published with `func azure functionapp publish` from the `backend/` directory, which is where `host.json`, `requirements.txt` and `function_app.py` live.

```
FUNCTION_APP_NAME="cpsy300-nutrition-api"
PYTHON_VERSION="3.11"

az functionapp create \
    --name "$FUNCTION_APP_NAME" \
    --resource-group "$RESOURCE_GROUP" \
    --storage-account "$STORAGE_ACCOUNT" \
    --consumption-plan-location "$LOCATION" \
    --runtime python \
    --runtime-version "$PYTHON_VERSION" \
    --functions-version 4 \
    --os-type Linux \
    --output none

az functionapp config appsettings set \
    --name "$FUNCTION_APP_NAME" \
    --resource-group "$RESOURCE_GROUP" \
    --settings \
        "AZURE_STORAGE_CONNECTION_STRING=$CONN_STRING" \
        "BLOB_CONTAINER_NAME=$BLOB_CONTAINER" \
        "BLOB_NAME=All_Diets.csv" \
    --output none

(cd backend && func azure functionapp publish "$FUNCTION_APP_NAME")
```

3.4 Static Web App and frontend build — steps 5 and 6 of 7

Azure Static Web App Deployment

The Static Web App is created **unlinked** from GitHub, so no repository token or OAuth grant is required. The frontend is then built into a throwaway `build/` directory with the live API URL substituted in, and uploaded with the Static Web Apps CLI using a deployment token read from Azure. If `swa` is not installed the script prints the exact command to run manually instead of failing silently.

```
STATIC_WEB_APP_NAME="cpsy300-nutrition-dashboard"

az staticwebapp create \
    --name "$STATIC_WEB_APP_NAME" \
    --resource-group "$RESOURCE_GROUP" \
    --location "$SWA_LOCATION" \
    --sku Free \
    --output none

SWA_HOSTNAME=$(az staticwebapp show --name "$STATIC_WEB_APP_NAME" \
    --resource-group "$RESOURCE_GROUP" --query defaultHostname --output tsv)
DASHBOARD_URL="https://${SWA_HOSTNAME}"

# Build into a separate directory so the committed sources keep their
# placeholder and the repository stays clean.
rm -rf build
cp -r frontend build
cp staticwebapp.config.json build/ 2>/dev/null || true
sed -i "s#__FUNCTION_API_URL__#${FUNCTION_URL}/api#" build/index.html
grep -q "${FUNCTION_URL}/api" build/index.html \
    || { echo "ERROR: failed to inject the API URL into index.html" >&2; exit 1; }

SWA_TOKEN=$(az staticwebapp secrets list --name "$STATIC_WEB_APP_NAME" \
    --resource-group "$RESOURCE_GROUP" --query properties.apiKey --output tsv)
swa deploy build --deployment-token "$SWA_TOKEN" --env production
```

3.5 CORS scoping and smoke test — step 7 of 7

> **CORS is scoped to exactly one origin**
>
> Step 7 first removes any `*` wildcard left behind by an earlier run, then allows only the Static Web App origin. It finishes by calling all three endpoints with `curl -fsS --max-time 180`, which both verifies the deployment and absorbs the cold start.

```
az functionapp cors remove \
    --name "$FUNCTION_APP_NAME" --resource-group "$RESOURCE_GROUP" \
    --allowed-origins "*" --output none 2>/dev/null || true

az functionapp cors add \
    --name "$FUNCTION_APP_NAME" --resource-group "$RESOURCE_GROUP" \
    --allowed-origins "$DASHBOARD_URL" --output none

for endpoint in health "insights" "recipes?page=1&limit=5"; do
    curl -fsS --max-time 180 "$FUNCTION_URL/api/$endpoint" >/dev/null && echo OK || echo FAILE
done
```

3.6 Environment & Configuration

These are the exact app-setting names read by `backend/function_app.py`. Local development uses the same names in `backend/local.settings.json`, which is why no code changes are needed between Azurite and Azure.

Setting	Value	Purpose
`AZURE_STORAGE_CONNECTION_STRING`	Connection string for `cpsy300nutrition<hex>`	Primary credential used by `BlobServiceClient`. Set by `deploy.sh`; never committed.
`BLOB_CONTAINER_NAME`	`datasets`	Private container holding the dataset. Code default is also `datasets`.
`BLOB_NAME`	`All_Diets.csv`	Blob to read, and the value reported in `metadata.dataset`.

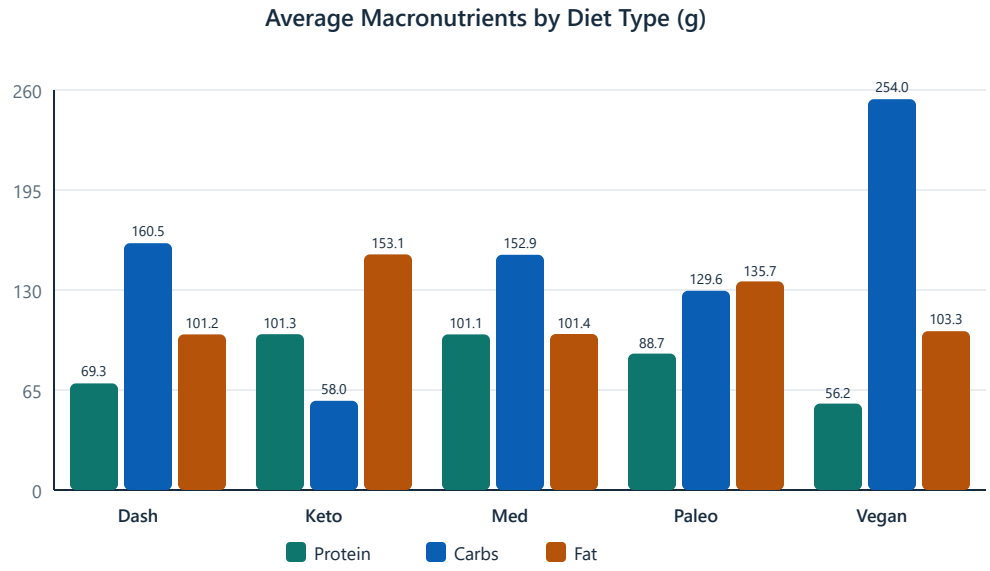
Setting	Value	Purpose
`AzureWebJobsStorage`	Created by `az functionapp create`	Required by the Functions host. Used as a fallback by `_get_connection_string()` if the explicit setting is absent; locally it is `UseDevelopmentStorage=true` via `local.settings.json`.
`FUNCTIONS_WORKER_RUNTIME`	`python`	Worker language. Runtime version 3.11, Functions version 4.
Host CORS allowed origin	The Static Web App origin, e.g. `https://cpsy300-nutrition-dashboard.<id>.azurestaticapps.net`	Set in step 7. The wildcard is explicitly removed first.
`host.json` → `extensions.http.routePrefix`	`api`	Why the routes resolve as `/api/health`, `/api/insights`, `/api/recipes`.
`__FUNCTION_API_URL__` (build placeholder)	Replaced at build time with `<function-url>/api`	Keeps the committed `index.html` free of environment-specific URLs.

4. Dashboard Visualizations

The dashboard renders four distinct visualizations from the `/api/insights` payload. The static SVGs below reproduce them with the verified figures recomputed from `Project1/data/All_Diets.csv` (7,806 recipes, 5 diet types), so the report and the dashboard show the same numbers. In the running dashboard the bar, doughnut and scatter charts are Chart.js canvases and the heatmap is a CSS grid built in `app.js`; all four re-render when the diet filter changes.

4.1 Average Macronutrients by Diet Type (Grouped Bar Chart)

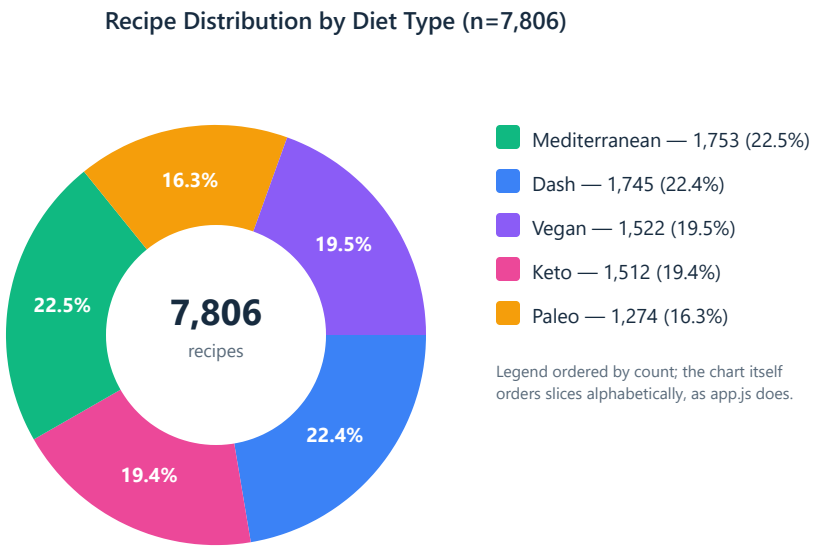
Chart.js grouped bar chart of mean Protein, Carbs and Fat per diet type.



Key insight: Vegan has by far the highest average carbs (254.0 g) and the lowest average protein (56.2 g). Keto is the mirror image: the lowest carbs (58.0 g) and the highest fat (153.1 g). Keto (101.27 g) and Mediterranean (101.11 g) are within 0.2 g of each other on mean protein, so keto takes `highest_mean_protein_diet` by a very small margin.

4.2 Recipe Distribution by Diet Type (Doughnut Chart)

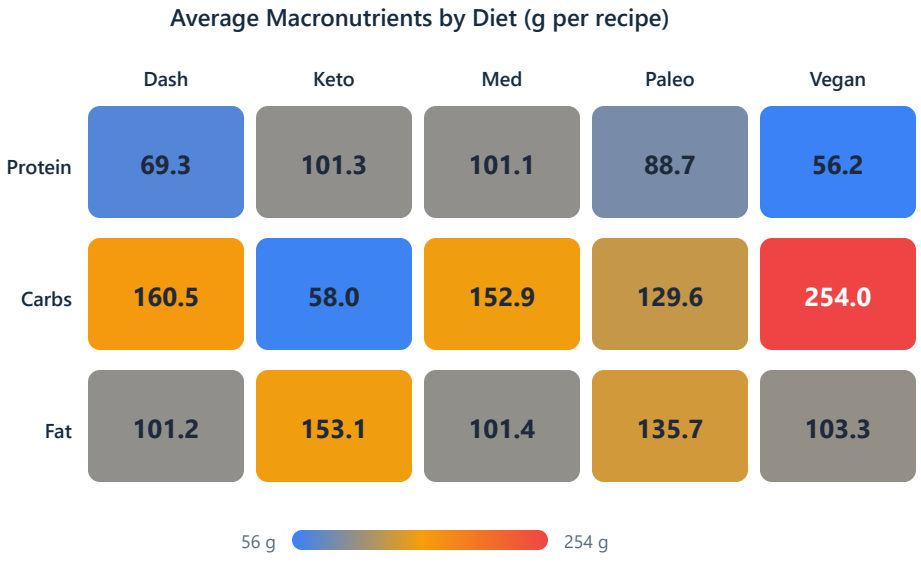
Chart.js doughnut of the 7,806 recipes across the 5 diet categories. Counts come from `summary.diet_counts`; the tooltip shows the count and the percentage.



Key insight: Mediterranean has the largest share at **22.5%** (1,753 recipes) and paleo the smallest at **16.3%** (1,274). Mediterranean and dash are nearly tied (1,753 vs 1,745), as are vegan and keto (1,522 vs 1,512). The dataset is therefore close to balanced: the gap between the largest and smallest category is only 6.1 percentage points, so no diet dominates the aggregate statistics.

4.3 Macronutrient Intensity Heatmap

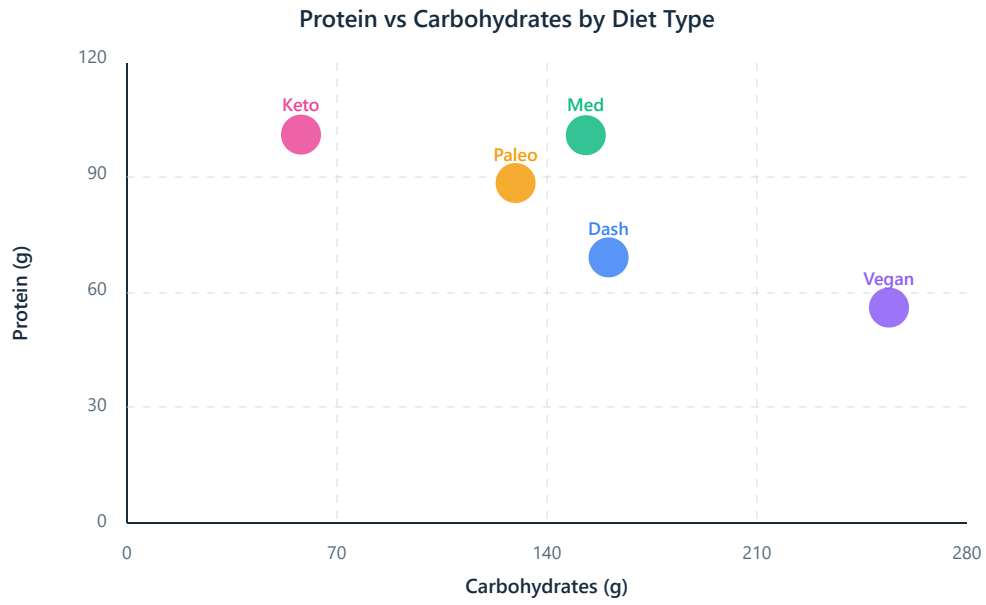
The dashboard heatmap is a CSS grid of average macro values: diets across the columns, Protein / Carbs / Fat down the rows. Cell colour runs blue (lowest) → amber → red (highest) over the observed range, exactly as `renderHeatmap()` computes it.



Key insight: The single hottest cell in the grid is vegan carbs (254.0 g) and the coldest is vegan protein (56.2 g), so the vegan column is the most extreme in both directions. Keto and paleo are the only diets whose average fat exceeds their average carbs (153.1 g vs 58.0 g for keto, 135.7 g vs 129.6 g for paleo). **Related statistic:** computed across all 7,806 individual recipes, the three macros are all *positively* correlated — Protein–Fat **+0.478**, Carbs–Fat **+0.269**, Protein–Carbs **+0.156**. Larger recipes simply contain more of everything; the low-carb/high-fat trade-off shows up between *diet means*, not between individual recipes.

4.4 Protein vs Carbs Scatter Plot

Chart.js scatter plot. Each point is one diet type's average carbs (x) against average protein (y), one dataset per diet so the legend can toggle them.



Key insight: Across diet averages the two macros trade off: keto sits upper-left (low carbs, high protein) and vegan lower-right (high carbs, low protein). Mediterranean and keto have practically the same mean protein (101.1 g vs 101.3 g) but differ by 95 g of carbs, which is what separates the two profiles. Note that this inverse pattern holds between diet means only — at the level of individual recipes protein and carbs are weakly *positively* correlated (+0.156).

Other dashboard panels fed by the same payload

Besides the four charts the page renders: six headline stat badges (total recipes, diet types, highest mean protein diet and value, highest total protein diet and value), a "Top 5 Highest-Protein Recipes per Diet" table, a "Most Common Cuisines" table, and a paged "Recipe Data" table. Verified values behind them:

Diet	Recipes	Mean protein (g)	Total protein (g)	Most common cuisine
mediterranean	1,753	101.11	**177,249.89**	mediterranean (1,274)
dash	1,745	69.28	120,897.57	american (639)
vegan	1,522	56.16	85,471.00	american (925)
keto	1,512	**101.27**	153,114.96	american (663)
paleo	1,274	88.67	112,971.65	american (535)

Highest mean protein: keto (101.27 g). **Highest total protein:** mediterranean (177,249.89 g) — it wins on volume, not on per-recipe density. The three highest-protein individual recipes are paleo "Swiss Paleo’s Homemade Italian & Chorizo…" (1,273.61 g), dash "Salmon Mousse" (1,239.47 g) and keto "Sara Louise's Keto Smoked Holiday Turkey" (1,092.00 g). Four of the five diets have *american* as their most common cuisine; mediterranean is the exception, which is expected given the label.

5. Key Code Sections

The excerpts below are copied from the repository; `...` marks elided lines only.

5.1 Azure Function — v2 programming model (`backend/function_app.py`)

There is no `function.json` anywhere in this project. Routes are declared in Python with the v2 decorator model: a single `func.FunctionApp` object with one `@app.route( ... )` per endpoint. The `api` prefix comes from `host.json`, so the routes resolve as `/api/health`, `/api/insights` and `/api/recipes`.

```
"""Azure Functions v2 HTTP API for Nutritional Insights Dashboard.

Endpoints:
    GET /api/health   -- liveness check (no storage call)
    GET /api/insights -- average macros, top recipes, common cuisines, summary stats
    GET /api/recipes  -- recipe rows; optional ?diet_type=, ?page=, ?limit=

CORS is configured at the Function App host level (see deploy.sh), so this
module deliberately does not emit Access-Control-Allow-* headers itself --
doing both produces duplicate headers, which browsers reject.
"""
import io, json, logging, os, threading, time

import azure.functions as func
import pandas as pd
from azure.storage.blob import BlobServiceClient

from nutrition import average_macros, clean_data, common_cuisines, top_recipes

app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

DEFAULT_PAGE_SIZE = 50
MAX_PAGE_SIZE = 500

RESPONSE_COLUMNS = ["Diet_type", "Recipe_name", "Cuisine_type",
                    "Protein(g)", "Carbs(g)", "Fat(g)"]

# Cleaned dataset cache, keyed by the blob ETag so a re-uploaded CSV is picked
# up automatically. A worker handles requests concurrently, hence the lock.
_cache = {"etag": None, "frame": None}
_cache_lock = threading.Lock()


def _load_dataset():
    """Return the cleaned dataset, downloading from Blob Storage on a cache miss.

    The blob ETag is checked on every request (a cheap metadata call) and the
    CSV is only re-downloaded and re-cleaned when it has actually changed.
    """
    container = os.environ.get("BLOB_CONTAINER_NAME", "datasets")
    blob_name = os.environ.get("BLOB_NAME", "All_Diets.csv")

    client = BlobServiceClient.from_connection_string(_get_connection_string())
    try:
        blob_client = client.get_blob_client(container=container, blob=blob_name)
        etag = blob_client.get_blob_properties().etag
```

```
        with _cache_lock:
            if _cache["etag"] == etag and _cache["frame"] is not None:
                return _cache["frame"], True

        data = blob_client.download_blob().readall()
        frame = clean_data(pd.read_csv(io.BytesIO(data)))

        with _cache_lock:
            _cache["etag"] = etag
            _cache["frame"] = frame
        return frame, False
    finally:
        client.close()


def _error_response(message, status_code=500):
    """Return a JSON error response.

    Internal exception text is logged, never returned, so storage details
    cannot leak to the browser.
    """
    return _json_response({"error": message}, status_code)


@app.route(route="health", methods=["GET"])
def health(req: func.HttpRequest) -> func.HttpResponse:
    """Liveness check. Does not touch storage, so it stays fast and cheap."""
    return _json_response({
        "status": "healthy",
        "service": "nutritional-insights-api",
        "version": "2.0.0",
        "dataset_cached": _cache["frame"] is not None,
    })


@app.route(route="insights", methods=["GET"])
def insights(req: func.HttpRequest) -> func.HttpResponse:
    """Return aggregate nutritional insights: averages, top recipes, common cuisines."""
    start = time.time()
    try:
        df, cache_hit = _load_dataset()

        avg = average_macros(df)                        # mean macros per diet
        top = top_recipes(df)                           # top 5 by protein per diet
        cuisine_records = common_cuisines(df).to_dict(orient="records")
```

```
        totals = df.groupby("Diet_type")["Protein(g)"].sum()
        diet_counts = {str(k): int(v) for k, v in df["Diet_type"].value_counts().items()}
        ...
        return _json_response({
            "average_macros": avg_records,
            "top_recipes": top_records,
            "common_cuisines": cuisine_records,
            "total_protein_by_diet": total_records,
            "summary": summary,
            "metadata": {
                "execution_time_seconds": round(time.time() - start, 3),
                "source": "Azure Blob Storage",
                "dataset": os.environ.get("BLOB_NAME", "All_Diets.csv"),
                "dataset_from_cache": cache_hit,
            },
        })
    except Exception:
        logging.exception("Error in /api/insights")
        return _error_response("Failed to compute insights from the dataset.")
```

5.2 Server-side pagination (`/api/recipes`)

The full recipe payload for all 7,806 rows is about **1.21 MB** of JSON, so `/api/recipes` pages on the server. `limit` defaults to 50 and is clamped to `MAX_PAGE_SIZE = 500`; `page` is clamped to the last available page; a non-numeric or non-positive value returns HTTP 400; an unknown `diet_type` returns HTTP 404. The response carries the paging metadata the frontend needs to draw its page buttons.

```
@app.route(route="recipes", methods=["GET"])
def recipes(req: func.HttpRequest) -> func.HttpResponse:
    """Return a page of recipe rows, optionally filtered by diet_type.

    Query parameters:
        diet_type -- filter to one diet ("all" or omitted returns every diet)
        page      -- 1-based page number (default 1)
        limit     -- rows per page, capped at MAX_PAGE_SIZE (default 50)
    """
    start = time.time()
    try:
        page = _positive_int(req.params.get("page"), 1)
        limit = _positive_int(req.params.get("limit"), DEFAULT_PAGE_SIZE, MAX_PAGE_SIZE)
    except ValueError as exc:
        return _error_response(f"Invalid pagination parameter: {exc}", 400)

    try:
        df, cache_hit = _load_dataset()

        diet_type = req.params.get("diet_type")
        if diet_type and diet_type.lower() != "all":
            df = df[df["Diet_type"].str.lower() == diet_type.lower()]
            if df.empty:
                return _error_response(f"No recipes found for diet type: {diet_type}", 40

        total = len(df)
        total_pages = max(1, -(-total // limit))        # ceiling division
        page = min(page, total_pages)
        offset = (page - 1) * limit

        records = df[RESPONSE_COLUMNS].iloc[offset:offset + limit].to_dict(orient="record

        return _json_response({
            "recipes": records,
            "count": len(records),
            "total": total,
            "page": page,
            "limit": limit,
            "total_pages": total_pages,
            "diet_type_filter": diet_type or "all",
            "metadata": {...},
        })
    except Exception:
```

```
        logging.exception("Error in /api/recipes")
        return _error_response("Failed to read recipes from the dataset.")
```

5.3 Frontend fetch and offline fallback (`frontend/app.js`)

The API base URL is read from `window.NUTRITIONAL_API_URL`, which `index.html` sets from the build-time placeholder. If `/api/insights` or `/api/recipes` fails, the whole dashboard switches to offline mode and renders the committed Project 1 batch results behind a prominent banner — live and fallback data are never mixed in one view.

```
// The deployed Function endpoint is injected by index.html via
// window.NUTRITIONAL_API_URL. "/api" only works when the dashboard and the
// Function are served from the same origin.
const API_BASE_URL = String(window.NUTRITIONAL_API_URL || "/api").replace(/\/+$/, "");

async function apiGet(path) {
    const response = await fetch(`${API_BASE_URL}${path}`);
    if (!response.ok) throw new Error(`API returned ${response.status}`);
    return response.json();
}

const fetchHealth   = () => apiGet("/health");
const fetchInsights = () => apiGet("/insights");

function fetchRecipes(dietType, page) {
    const params = new URLSearchParams({ page: String(page), limit: String(ROWS_PER_PAGE)
    if (dietType && dietType !== "all") params.set("diet_type", dietType);
    return apiGet(`/recipes?${params.toString()}`);
}

async function loadData(dietType, page = 1) {
    hideError();
    setLoadingAll(true);

    let health = null;
    try { health = await fetchHealth(); }
    catch (err) { console.warn("Health check failed:", err.message); }

    try {
        const data = await fetchInsights();
        const recipesResp = await fetchRecipes(dietType, page);

        insightsData = data;
        isOffline = false;

        recipeState.rows       = recipesResp.recipes || [];
        recipeState.page       = recipesResp.page || 1;
        recipeState.totalPages = recipesResp.total_pages || 1;
        recipeState.total      = recipesResp.total ?? recipeState.rows.length;
        recipeState.serverPaged = true;                // paging served by the Function

        updateApiStatus(true, health);
        updateExecTime(data.metadata?.execution_time_seconds);
        setOfflineBanner(null);
        $("#data-source-label").textContent = "Source: Azure Blob Storage (live)";
    } catch (err) {
```

```
        console.warn("API unavailable, using offline fallback:", err.message);
        insightsData = DEMO_DATA;                        // Project 1 batch results
        isOffline = true;
        loadOfflineRecipes(dietType, 1);

        updateApiStatus(false, health);
        updateExecTime(null);
        setOfflineBanner(`Could not reach ${API_BASE_URL} (${err.message}).`);
        $("#data-source-label").textContent = "Source: offline fallback";
    }

    renderAll(dietType);
    setLoadingAll(false);
}
```

5.4 Configuration: route prefix, local settings, build-time URL injection

Three files carry all the environment coupling. `host.json` sets the `api` route prefix; `local.settings.json.example` mirrors the production app-setting names so the same code runs against Azurite (the real `local.settings.json` is generated from it locally and is git-ignored, so no connection string is ever committed); and `index.html` holds a placeholder that `deploy.sh` rewrites into the build directory.

```
// backend/host.json
{
    "version": "2.0",
    "logging": { "applicationInsights": { "samplingSettings": { "isEnabled": true, "exclu
    "extensionBundle": { "id": "Microsoft.Azure.Functions.ExtensionBundle", "version": "[
    "extensions": { "http": { "routePrefix": "api" } }
}

// backend/local.settings.json.example (template for local development; never deployed)
{
    "IsEncrypted": false,
    "Values": {
        "AzureWebJobsStorage": "UseDevelopmentStorage=true",
        "FUNCTIONS_WORKER_RUNTIME": "python",
        "AZURE_STORAGE_CONNECTION_STRING": "UseDevelopmentStorage=true",
        "BLOB_CONTAINER_NAME": "datasets",
        "BLOB_NAME": "All_Diets.csv"
    },
    "Host": { "CORS": "*", "CORSCredentials": false }
}
```

```
<!-- frontend/index.html -- the committed source keeps a placeholder, never a real URL --
<script>
    window.NUTRITIONAL_API_URL = "__FUNCTION_API_URL__";
    if (window.NUTRITIONAL_API_URL.startsWith("__")) {
        // Placeholder was never replaced -- fall back to a same-origin /api.
        delete window.NUTRITIONAL_API_URL;
    }
</script>
```

The wildcard `Host.CORS` above applies to `func start` on localhost only, and `local.settings.json` is never published to Azure. In Azure, CORS is set at the host level by `deploy.sh` step 7 to the Static Web App origin alone.

6. Frontend-Backend Integration

Data Flow



```
On failure at step 3 or 8: the whole page switches to the offline banner and renders
Project1/docs/results/*.csv values. Live and fallback data are never mixed.
```

Integration Points

Step	Component	Action	Technology
1	Static Web App	Serves `index.html`, `app.js`, `styles.css` with the API URL already substituted into the page	Azure Static Web Apps (Free), `staticwebapp.config.json`
2	Browser → Function	`GET /api/health` — dependency-free liveness probe; drives the "API Connected (v2.0.0)" badge and reports `dataset_cached`	JavaScript Fetch API
3	Browser → Function	`GET /api/insights` for averages, top recipes, cuisines, totals and summary	JavaScript Fetch API
4-5	Function → Storage	Reads the blob ETag; re-downloads `All_Diets.csv` only when the ETag changed, otherwise serves the cached DataFrame	`azure-storage-blob` SDK, `threading.Lock`
6	Function	`clean_data()` then `average_macros()` / `top_recipes()` / `common_cuisines()` plus totals and counts	pandas, shared `nutrition.py`
7	Function → Browser	JSON response including `metadata.execution_time_seconds` and `metadata.dataset_from_cache`	HTTP 200, `application/json`
8	Browser → Function	`GET /api/recipes?diet_type=&page=&limit=50` for the paged recipe table; page buttons re-fetch from the server	Fetch API + `URLSearchParams`
9	Browser	Renders 4 charts, 3 tables and 6 stat badges; shows the insights execution time in the header badge	Chart.js 4.4.1 + CSS-grid heatmap

Frontend Controls

Control	Element	Behaviour
Diet-type dropdown	`#diet-filter`	Six options (All, Dash, Keto, Mediterranean, Paleo, Vegan). On `change` it reloads insights and page 1 of the recipes for that diet, then re-renders the bar, heatmap and scatter charts plus both detail tables. The doughnut and the stat badges deliberately keep showing the whole dataset.
Refresh Data button	`#refresh-btn`	Re-runs the same load for the current filter, so a marker can demonstrate a live call on demand and watch the execution-time badge change.
Pagination controls	`#pagination`	Prev / numbered / Next buttons with ellipsis compression, built from `total_pages`. Clicks are handled by one delegated listener and fetch the next page from the Function (`serverPaged`); offline they slice the fallback rows in the browser.
API status badge	`#api-status`	Green "API Connected (v<version>)" from `/api/health`, or amber "Offline - fallback data".
Execution-time badge	`#exec-time-display`	Shows `metadata.execution_time_seconds` from the `/api/insights` call.
Offline banner	`#offline-banner`	Full-width banner naming the unreachable URL and stating that every figure below comes from the committed `Project1/docs/results/` CSV files.

There is deliberately **no free-text search box and no per-endpoint button row**; the dropdown, the refresh button and the pagination controls are the complete set of interactive controls.

CORS Configuration

One layer of CORS, scoped to one origin

CORS is handled **only** at the Function App host level. `deploy.sh` step 7 removes any `*` wildcard from an earlier run and then allows the Static Web App origin alone. `backend/function_app.py` deliberately emits no `Access-Control-Allow-*` headers of its own: if the host and the application both set them the browser receives duplicate headers and rejects the response, so the application layer stays out of it. The design decision is recorded in the module docstring.

```
az functionapp cors remove \
  --name cpsy300-nutrition-api \
  --resource-group cpsy300-nutritional-insights-rg \
  --allowed-origins "*" --output none 2>/dev/null || true

az functionapp cors add \
  --name cpsy300-nutrition-api \
  --resource-group cpsy300-nutritional-insights-rg \
  --allowed-origins "$DASHBOARD_URL"
```

The dashboard sends no custom request headers, so no preflight is triggered and no route needs to accept `OPTIONS`. Verified in Azure — `az functionapp cors show` lists the dashboard origin as the only allowed origin, and a live call returns `Access-Control-Allow-Origin` for that origin (Figure 7).

API Endpoints

Endpoint	Method	Parameters	Response
`/api/health`	GET	None	`status`, `service`, `version` (2.0.0) and `dataset_cached`. Touches no storage, so it stays fast and cheap and works even if the blob is unreachable.
`/api/insights`	GET	None	`average_macros`, `top_recipes` (top 5 by protein per diet), `common_cuisines`, `total_protein_by_diet`, `summary` (totals, diet counts, highest mean/total protein) and `metadata` (`execution_time_seconds`, `dataset_from_cache`). About 5.8 KB of JSON.

Endpoint	Method	Parameters	Response
`/api/recipes`	GET	`?diet_type=<diet>&page=<n>&limit=<n>` — all optional. `limit` defaults to 50 and is capped at 500; `page` defaults to 1 and is clamped to the last page; `diet_type=all` or omitted returns every diet.	Paged recipe rows plus `count`, `total`, `page`, `limit`, `total_pages`, `diet_type_filter` and `metadata`. Invalid paging returns HTTP 400; an unknown diet returns HTTP 404.

These three routes are the whole API surface. There is no clustering, segmentation or machine-learning endpoint in this project.

> **Why paging exists, and what the cache saves**
>
> Serialising all 7,806 cleaned rows produces roughly **1.21 MB** of JSON, which is slow to transfer on a cold Consumption-plan instance and visibly janky to render as table rows. With the default `limit=50` a page is about 1/156th of that. The ETag-keyed cache is the second half of the same idea: the CSV is downloaded and cleaned once per worker, then every later request pays only one cheap `get_blob_properties()` metadata call. `metadata.dataset_from_cache` reports which path a given request took, so the effect is visible in the response rather than merely claimed, and `/api/health` exposes the same state as `dataset_cached`.

> **Error handling: generic to the browser, detailed in the logs**
>
> Both data endpoints wrap their body in `try/except Exception`, call `logging.exception( ... )`, and return a fixed message such as `{"error": "Failed to compute insights from the dataset."}`. The exception text — which can contain the storage account name, container, blob path or SDK internals — never reaches the client. The detail is still available in Application Insights, which `host.json` enables.

7. Rubric & Status

Every status below is backed by live evidence captured on 2026-10-20 (see Deployment Evidence below).

Category	Weight	Requirements	What exists in the repository	Status
Deployment	20 marks	Deploy Azure Function and Storage in the Azure cloud environment	`deploy.sh` creates the resource group, StorageV2 account, private `datasets` container, dataset upload, Consumption-plan Function App on Python 3.11, app settings, Static Web App and CORS scoping in seven idempotent steps, then smoke-tests all three endpoints. **Run on 2026-10-20**: resource group, storage account, Function App and Static Web App all exist; see Figures 2–5.	Done
Frontend Dashboard	20 marks	Functional and visually clear dashboard UI	`frontend/index.html` + `app.js` + `styles.css`: responsive Tailwind layout, header with API-status and execution-time badges, six stat badges, four chart cards, two detail tables, a paged recipe table, per-chart loading spinners, an error banner and an offline banner. Controls are a diet dropdown, a Refresh Data button and server-backed pagination.	Done
Data Visualization	20 marks	At least 3 distinct visualizations	Four types, all driven by the live `/api/insights` payload: grouped bar (average macros), doughnut (recipe distribution), CSS-grid heatmap (macro intensity, blue→amber→red with its own legend) and scatter (protein vs carbs, one dataset per diet). All re-render on filter change. Offline they render the committed Project 1 batch results behind an explicit banner.	Done

Category	Weight	Requirements	What exists in the repository	Status
Integration	20 marks	Frontend fetches and displays data from the Azure Function endpoint	`fetch()` against `/api/health`, `/api/insights` and `/api/recipes`; the endpoint is injected at build time from the `__FUNCTION_API_URL__` placeholder so the committed source carries no environment URL. Paging, the diet filter and the execution-time badge all round-trip through the API. **Demonstrated end to end**: the public dashboard reads live data from the Function App and shows "API Connected (v2.0.0)" with the execution time (Figures 1, 6–7).	Done
Cloud Practices	10 marks	Proper use of Azure resources (resource group, env vars, storage connection)	One resource group for everything; connection string only ever in app settings, never committed and never echoed by the script; private container with public blob access disabled and TLS 1.2 minimum; CORS scoped to a single origin with the wildcard explicitly removed; ETag-keyed cache to avoid re-downloading the blob; generic error bodies with detail only in the logs; security headers via `staticwebapp.config.json`; `local.settings.json` git-ignored with a committed `.example` template; unit tests and `flake8` enforced in CI. Verified against the live resources in Figures 4–5.	Done
Documentation & Presentation	10 marks	Architecture, workflow and challenges explanation	This overview (architecture, setup commands, data flow, code, endpoints, challenges), `Project2/README.md`, `docs/phase2-analysis.md` and the review under `Project2/docs/reviews/`. Deployment screenshots are in the Deployment Evidence section below; the submission PDF is exported from this page.	Done

Deployment Evidence (captured 2026-10-20)

Nutritional Insights Dashboard

CPSY 300 -- Phase 2: Cloud Dashboard Development

36 ms | API Connected (v2.0.0)

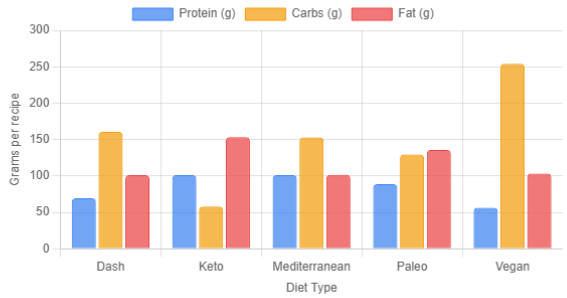
Filter by Diet: All Diet Types | Refresh Data

Source: Azure Blob Storage (live)

7,806	5	Keto	101.27g	Mediterranean	177,249.89g
TOTAL RECIPES	DIET TYPES	HIGHEST MEAN PROTEIN	MEAN PROTEIN (G)	HIGHEST TOTAL PROTEIN	TOTAL PROTEIN (G)

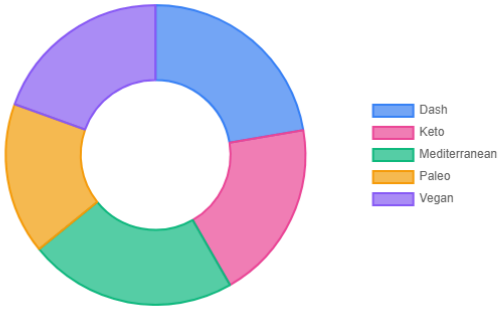
Average Macronutrients by Diet

Mean protein, carbs, and fat (grams per recipe) for each diet type.



Recipe Distribution by Diet

Number of recipes in each diet category.



Macronutrient Heatmap

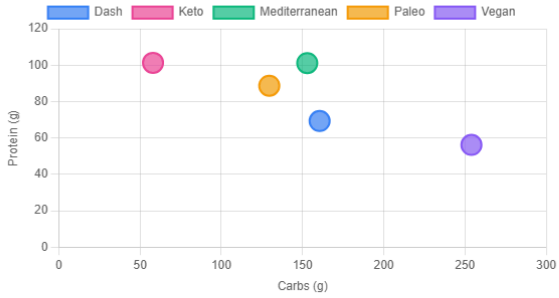
Color-coded intensity of average macronutrient values across diets.

	Dash	Keto	Mediterranean	Paleo	Vegan
Protein	69.3	101.3	101.1	88.7	56.2
Carbs	160.5	58.0	152.9	129.6	254.0
Fat	101.2	153.1	101.4	135.7	103.3

56g 254g

Protein vs Carbs by Diet

Average protein plotted against average carbs for each diet type.



Top 5 Highest-Protein Recipes per Diet

The five recipes with the most protein in each diet category.

DIET	RECIPE	CUISINE	PROTEIN (G)	CARBS (G)	F (G)
Dash	Salmon Mousse	Nordic	1239.5	22.4	9
Dash	Homemade Turkey Alphabet Soup	American	1190.3	49.1	2
Dash	Barbecue Chicken Legs	Mediterranean	1017.3	241.9	1
Dash	12th Man Hot Wings	American	807.0	49.2	6
Dash	Jazzed Up Turkey	American	517.5	575.6	

Most Common Cuisines

The most frequently appearing cuisine type for each diet.

DIET TYPE	CUISINE	RECIPE COUNT
Dash	American	639
Keto	American	663
Mediterranean	Mediterranean	1,274
Paleo	American	535
Vegan	American	925

Recipe Data

Browse recipes. Use the diet filter above to narrow results.

7,806 recipes (page 1 of 157)

DIET	RECIPE NAME	CUISINE	PROTEIN (G)	CARBS (G)	FAT (G)
Paleo	Bone Broth From 'Nom Nom Paleo'	American	5.2	1.3	3.2

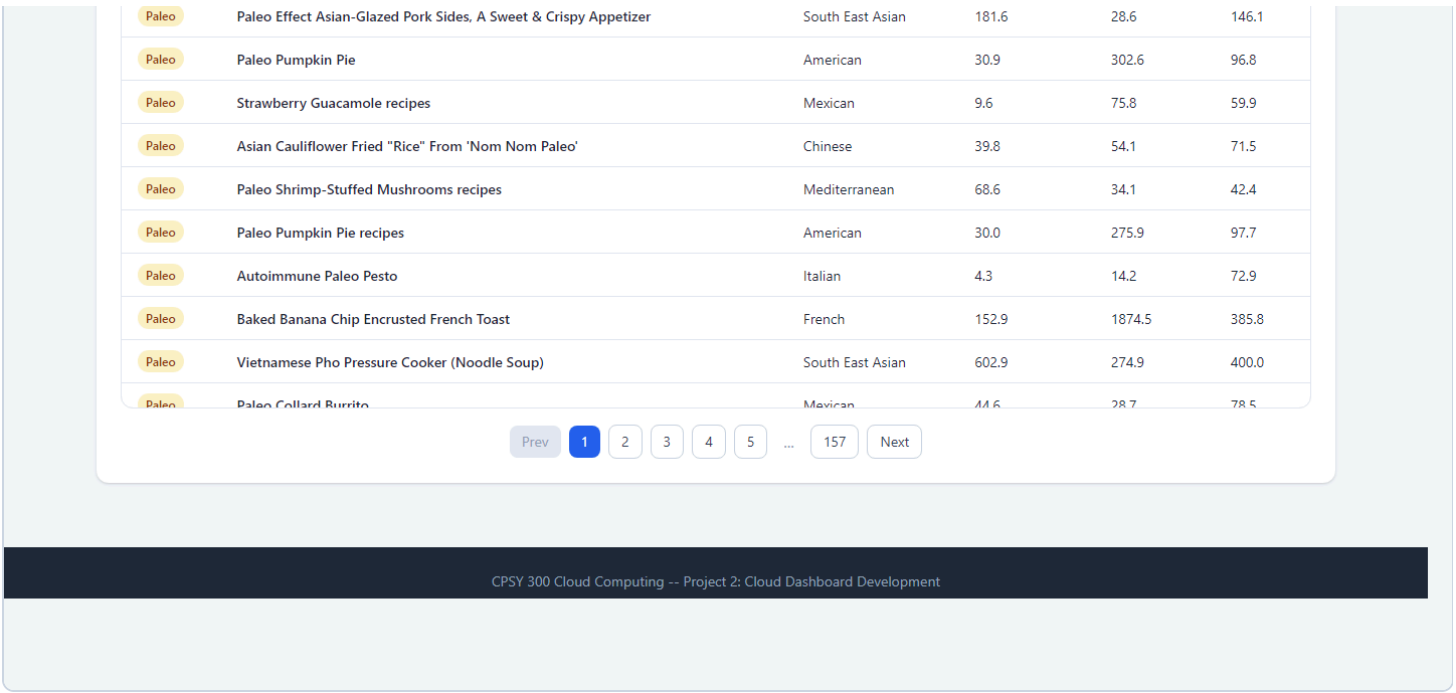


Figure 1. Public dashboard at https://salmon-ocean-0ce94150f.4.azurestaticapps.net: "API Connected (v2.0.0)" badge, execution-time badge, diet filter and Refresh button, bar, doughnut, heatmap and scatter charts and tables, all rendered from live Function App data.

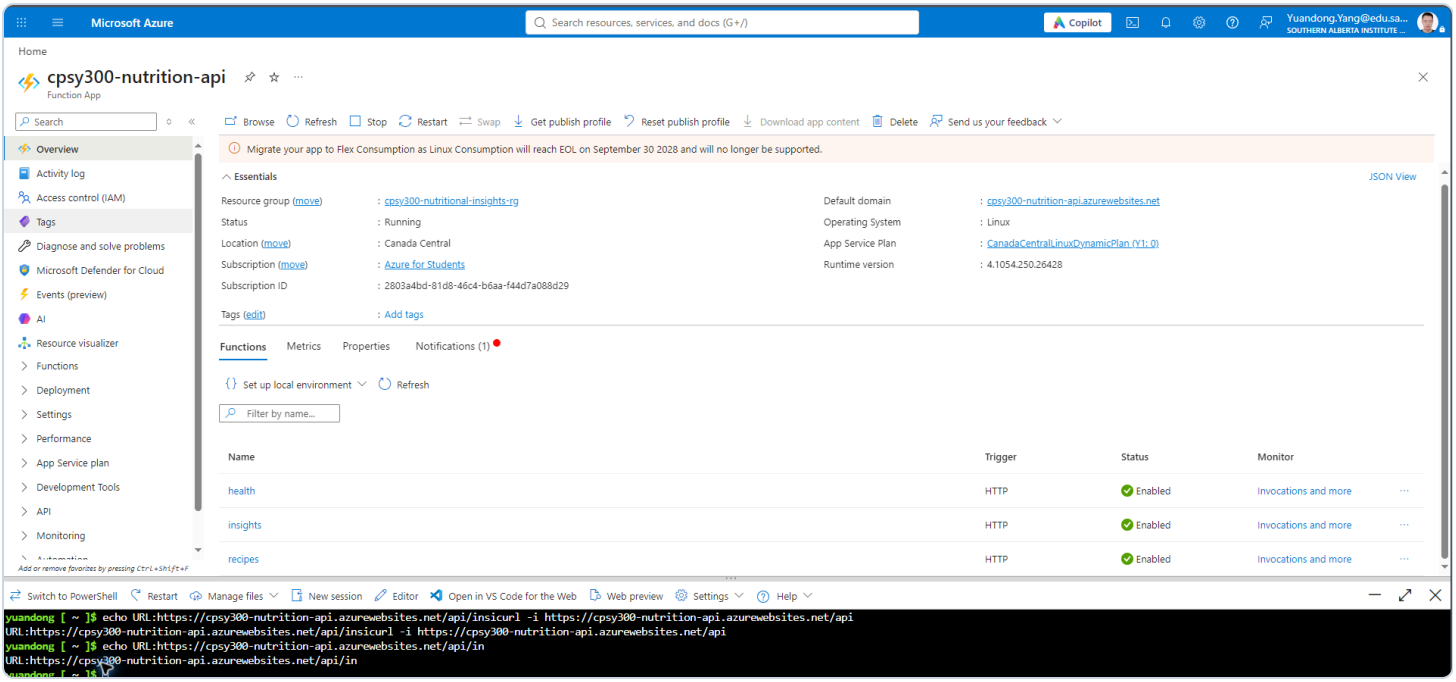


Figure 2. Azure Portal, Function App `cpsy300-nutrition-api`: status *Running*, Canada Central, Linux Consumption plan, default domain cpsy300-nutrition-api.azurewebsites.net, and the HTTP functions `health`, `insights` and `recipes`, all *Enabled*.

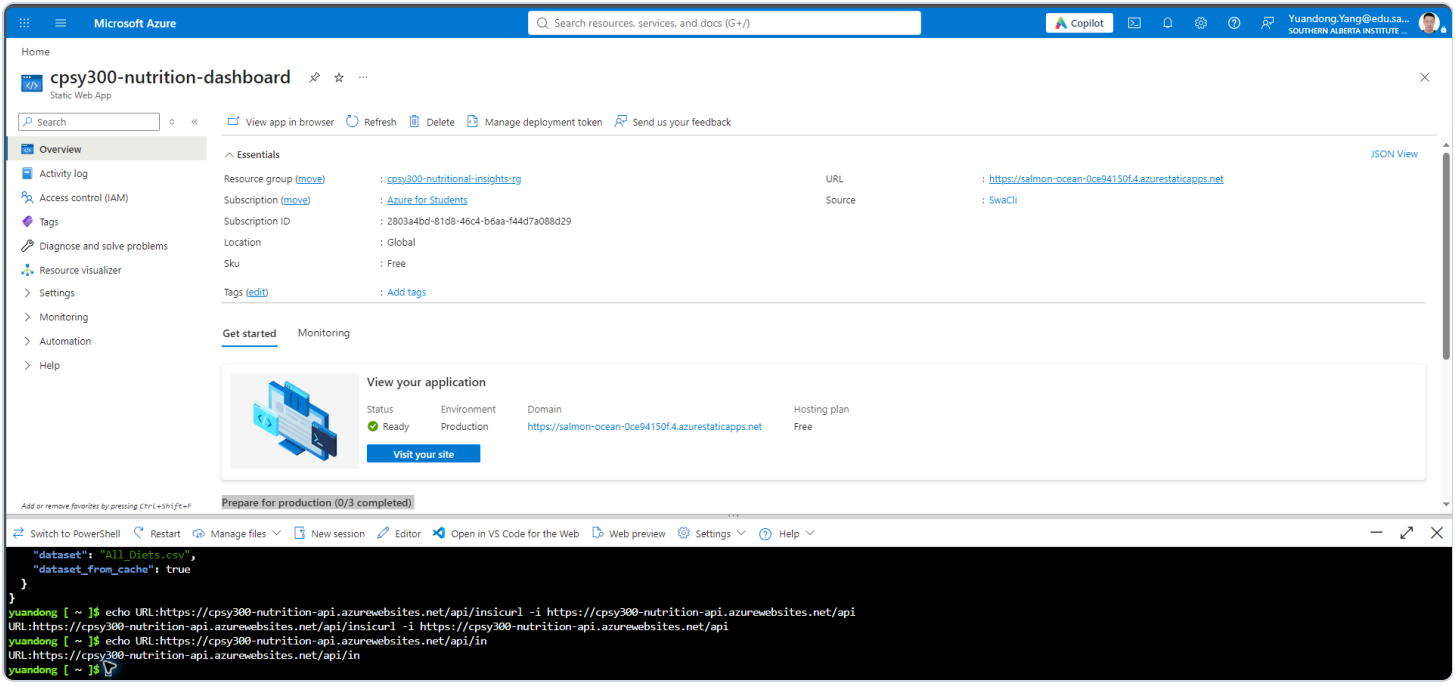


Figure 3. Azure Portal, Static Web App `cpsy300-nutrition-dashboard` (Free plan, status *Ready*, environment Production) at https://salmon-ocean-0ce94150f.4.azurestaticapps.net.



Azure resources in resource group cpsy300-nutritional-insights-rg

Captured: 2026-10-20 16:53:20 -06:00

Name	Type	Location
cpsy300nutritionf6a445	Microsoft.Storage/storageAccounts	canadacentral
cpsy300-nutrition-api	Microsoft.Web/sites	canadacentral
CanadaCentralLinuxDynamicPlan	Microsoft.Web/serverFarms	canadacentral
cpsy300-nutrition-api	Microsoft.Insights/components	canadacentral
cpsy300-nutrition-dashboard	Microsoft.Web/staticSites	eastus2

App settings (AZURE_STORAGE_CONNECTION_STRING exists but is hidden here):

Name	Value
FUNCTIONS_WORKER_RUNTIME	python
BLOB_CONTAINER_NAME	datasets
BLOB_NAME	All_Diets.csv

Blob container 'datasets' (public access: None (private)):

Name	Bytes	Modified
All_Diets.csv	702493	2026-10-20T22:48:47+00:00

Figure 4. Resources in resource group `cpsy300-nutritional-insights-rg` (Storage account, Function App, Consumption plan, Application Insights, Static Web App), the app settings used instead of hard-coded values, and the private `datasets` blob container holding All_Diets.csv. The connection string is kept in app settings and deliberately not shown.

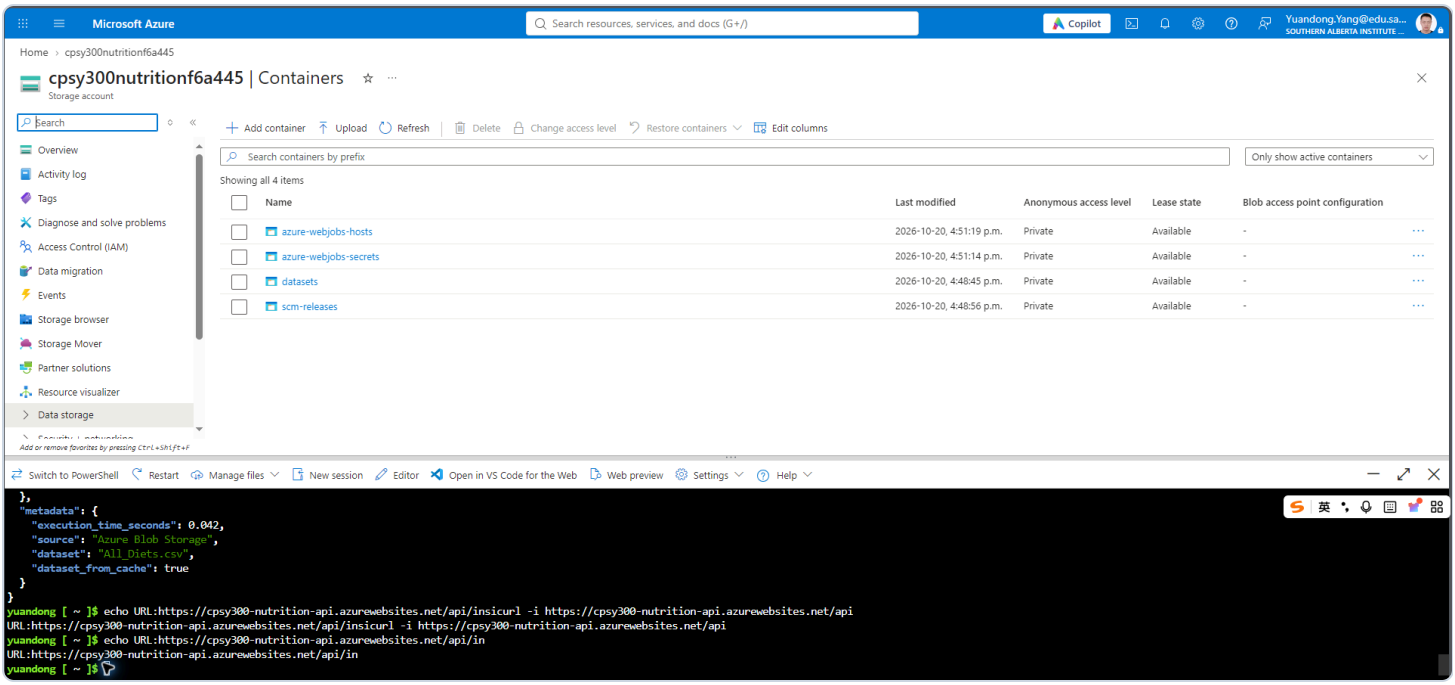


Figure 5. Azure Portal, storage account `cpsy300nutritionf6a445`: the `datasets` container has anonymous access level *Private*. Cloud Shell shows the `/api/insights` metadata: `execution_time_seconds` 0.042, source Azure Blob Storage, `dataset_from_cache` true.

```
{
    "average_macros": [
        {
            "Diet_type": "dash",
            "Protein(g)": 69.28,
            "Carbs(g)": 160.54,
            "Fat(g)": 101.15
        },
        {
            "Diet_type": "keto",
            "Protein(g)": 101.27,
            "Carbs(g)": 57.97,
            "Fat(g)": 153.12
        },
        {
            "Diet_type": "mediterranean",
            "Protein(g)": 101.11,
            "Carbs(g)": 152.91,
            "Fat(g)": 101.42
        },
        {
            "Diet_type": "paleo",
            "Protein(g)": 88.67,
            "Carbs(g)": 129.55,
            "Fat(g)": 135.67
        },
        {
            "Diet_type": "vegan",
            "Protein(g)": 56.16,
            "Carbs(g)": 254,
            "Fat(g)": 103.3
        }
    ],
    "top_recipes": [
        {
            "Diet_type": "dash",
            "Recipe_name": "Salmon Mousse",
            "Cuisine_type": "nordic",
            "Protein(g)": 1239.47,
            "Carbs(g)": 22.4,
            "Fat(g)": 983.36
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "Homemade Turkey Alphabet Soup",
            "Cuisine_type": "american",
            "Protein(g)": 1190.35,
            "Carbs(g)": 49.13,
            "Fat(g)": 297.45
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "Barbecue Chicken Legs",
            "Cuisine_type": "mediterranean",
            "Protein(g)": 1017.25,
            "Carbs(g)": 241.93,
            "Fat(g)": 1002.07
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "12th Man Hot Wings",
```


Figure 6. Browser test of https://cpsy300-nutrition-api.azurewebsites.net/api/insights, the JSON payload the dashboard consumes.



Figure 7. CORS restricted to the dashboard origin only, and a timed call to `/api/insights` (HTTP 200, 0.51 s including cold start). The response carries `Access-Control-Allow-Origin` for the dashboard origin.

Deliverables Checklist

Deliverable	Details	Status
Deployed Azure Function URL	https://cpsy300-nutrition-api.azurewebsites.net/api/insights (also `/api/health`, `/api/recipes`)	Done
Static Web App link	https://salmon-ocean-0ce94150f.4.azurestaticapps.net	Done
GitHub repository (frontend + backend)	github.com/YUANDONG-YANG/cloud-computing — all Project 2 code is committed under `Project2/`	Done
Azure portal screenshots	Resource group, Function App overview, app settings, blob container, Static Web App, and the dashboard showing "API Connected" — Figures 1–7 below	Done
Documentation PDF	Architecture diagram, services, workflow and screenshots — exported from this page to `Project2/docs/reports/project2-master-overview.pdf` (36 pages, background graphics enabled)	Done

Both URLs above were tested on 2026-10-20 and returned HTTP 200.

8. Challenges & Solutions

A challenge is marked Solved only when something in the repository implements the fix *and* the 2026-10-20 deployment evidence confirms it. Where nothing implements it, it is marked Open rather than quietly dropped.

#	Challenge	Solution	Outcome / evidence	
1	**Duplicate CORS headers** Setting `Access-Control-Allow-Origin` in the function code *and* at the Function App host level makes Azure emit the header twice, which browsers reject outright.	Keep exactly one layer. `function_app.py` emits no `Access-Control-Allow-*` headers at all (stated in its docstring) and `deploy.sh` step 7 removes any `*` wildcard before allowing only the Static Web App origin.	Confirmed against the live deployment: `az functionapp cors show` returns the dashboard origin alone, and `/api/insights` answers HTTP 200 with a single `Access-Control-Allow-Origin` header for that origin (Figure 7).	Solved
2	**Environment-specific URL in committed source** Hard-coding the Function URL in `index.html` would leak one team member's deployment into the repository and break for everyone else.	`index.html` ships the placeholder `__FUNCTION_API_URL__` and deletes `window.NUTRITIONAL_API_URL` if it was never substituted. `deploy.sh` copies `frontend/` to a throwaway `build/`, rewrites the placeholder with `sed -i "s#__FUNCTION_API_URL__# ... #"`, and `grep`s to confirm the substitution before deploying.	Committed sources stay environment-agnostic and the build fails loudly rather than shipping a placeholder.	Solved

#	Challenge	Solution	Outcome / evidence	
3	**Re-downloading the CSV on every request** Fetching and re-cleaning 7,806 rows per request wastes egress and adds latency to every call.	Cache the cleaned DataFrame keyed by the blob **ETag** under a `threading.Lock`. Each request makes one cheap `get_blob_properties()` call; the CSV is re-downloaded only when the ETag changed, so a re-uploaded dataset is still picked up automatically.	Repeat calls skip download and cleaning. `metadata.dataset_from_cache` and `/api/health`'s `dataset_cached` make the behaviour observable rather than assumed.	Solved
4	**Large JSON response size** The full recipe payload for all 7,806 rows measures about 1.21 MB, which is slow to transfer and laggy to render as table rows.	Server-side paging on `/api/recipes`: `?diet_type=&page=&limit=`, default `limit=50`, hard cap 500, with `count`, `total`, `page`, `limit` and `total_pages` in the response. Bad input returns HTTP 400; an unknown diet returns 404. The frontend's Prev/Next/numbered buttons re-fetch from the server.	A default page is roughly 1/156th of the full payload. Chart endpoints stay separate and return only ~5.8 KB of aggregates.	Solved
5	**Leaking internals through error responses** Returning `str(e)` to the browser would expose the storage account, container, blob path and SDK internals on any failure.	Every handler catches broadly, calls `logging.exception(...)`, and returns a fixed generic message such as `"Failed to compute insights from the dataset."` Detail goes to Application Insights, enabled in `host.json`.	No storage detail can reach a client. Diagnostics are still available to the team in the logs.	Solved

#	Challenge	Solution	Outcome / evidence	
6	**A dead API looking like a working dashboard** If the Function is unreachable, silently falling back to bundled data would show a marker a complete dashboard built from numbers that never came from the cloud.	A failure on either data call switches the *entire* page to offline mode: a prominent full-width banner names the unreachable URL, the status badge turns amber, the source label reads "offline fallback", and the figures shown are the committed Project 1 batch results from `Project1/docs/results/*.csv`. Live and fallback data are never mixed in one view.	Offline mode is unmistakable. The captured live dashboard is connected; this fallback remains available for API outages.	Solved
7	**Cold-start latency** A Consumption-plan Function can take several seconds to answer its first request, during which the dashboard would show empty chart frames.	Per-chart loading spinners plus a dependency-free `/api/health` probe that answers without touching storage; the `deploy.sh` smoke test uses `curl --max-time 180` so the first call absorbs the cold start instead of failing the script.	Measured on the live deployment: a cold `/api/insights` call returned HTTP 200 in 0.51 s (Figure 7), and once warm the dashboard reports 36 ms with `execution_time_seconds` 0.042 (Figures 1 and 5).	Solved
8	**Automated Azure deployment from CI** Pushing to `main` does not deploy anything to Azure.	**Not implemented.** `.github/workflows/deploy.yml` runs the Project 1 build/test/Docker simulation, and `.github/workflows/project2-tests.yml` runs lint and unit tests only. Deploying from CI would require a publish profile or service principal stored as a GitHub secret, which this project deliberately avoids; deployment is therefore a manual `./deploy.sh` run.	No deployment job, no Azure credentials in CI. The trade-off (no secrets in the repository vs. no push-to-deploy) is a conscious one, but the automation is genuinely absent.	Open

#	Challenge	Solution	Outcome / evidence	
9	**Static Web App region availability** `az staticwebapp create` fails in `canadacentral`, which is the region used for every other resource.	Use a separate `SWA_LOCATION=eastus2` for the Static Web App only, with the reason recorded in a comment next to the variable. The app is also created *unlinked* from GitHub so no repository token or OAuth grant is needed, and the frontend is uploaded afterwards with the `swa` CLI using a deployment token read from Azure.	The script can create every resource without interactive GitHub authorisation. If the `swa` CLI is missing it prints the exact manual command instead of failing silently.	Solved

9. Team Contributions

Team Member	Responsibilities	Key Contributions
Yuandong Yang	Project lead, deployment automation, CI, documentation	Wrote `deploy.sh`: the seven-step idempotent provisioning flow, private storage account with public blob access disabled and TLS 1.2 minimum, app-setting wiring that keeps the connection string out of the repository and the terminal, build-time API-URL injection into a throwaway `build/` directory, single-origin CORS scoping, and the endpoint smoke test. Added the `project2-tests` CI job (pytest, flake8 and a `bash -n deploy.sh` syntax check) and maintains this overview. *Deployment and Portal evidence captured on 2026-10-20.*
Ethan Bayarsaikhan	Azure Function development, data pipeline	Built the Azure Functions v2 app: `/api/health` (dependency-free liveness probe reporting `dataset_cached`), `/api/insights` (averages, top-5 protein recipes per diet, common cuisines, totals, summary stats and timing metadata) and `/api/recipes` (diet filter plus validated server-side paging with a 500-row cap). Added the ETag-keyed DataFrame cache with its lock, the generic-error/log-the-detail policy, and the `host.json` route prefix. Adapted Project 1's `nutrition.py` cleaning and aggregation so batch and API results agree.
Justin Norman-Rance	Frontend dashboard, data visualization	Built the responsive Tailwind dashboard and all four visualizations: Chart.js grouped bar, doughnut and scatter charts plus the hand-built CSS-grid heatmap with its blue→amber→red scale and legend. Implemented the diet-type dropdown, the Refresh Data button and the delegated pagination controls that page through the API, the six stat badges, the two detail tables, the API-status and execution-time badges, and the explicit offline banner that falls back to the committed Project 1 batch results.

Source Repository

What	Link
Repository	https://github.com/YUANDONG-YANG/cloud-computing
Project 2 folder	https://github.com/YUANDONG-YANG/cloud-computing/tree/main/Project2
Backend (Functions v2 app)	`Project2/backend/function_app.py`, `Project2/backend/nutrition.py`, `Project2/backend/host.json`
Frontend	`Project2/frontend/index.html`, `app.js`, `styles.css`, `Project2/staticwebapp.config.json`
Deployment script	`Project2/deploy.sh`
Phase 2 specification	spec/phase2-requirements.docx
Dashboard UI reference	spec/ui-skeleton.html
Data ground truth	`Project1/data/All_Diets.csv` and `Project1/docs/results/*.csv`

All figures recomputed from All_Diets.csv (7,806 recipes). Azure deployment evidence was captured on 2026-10-20; see the embedded Portal and live-dashboard screenshots.

Yuandong Yang • Ethan Bayarsaikhan • Justin Norman-Rance